CS 6158 Proposal

Mutation-Guided Regression Testing for Long-Horizon Coding Agents

Group Submission: Shipeng Zhao (sz825), Hengyu Zhang (hz827)

A. Problem

Mutation testing evaluates a test suite by introducing small artificial faults into a program and measuring how many are detected by the tests. Prior work has shown that mutation score is correlated with detection of real developer faults even after controlling for code coverage [1]. More recently, mutation-guided LLM systems have used surviving mutants to generate additional tests that strengthen existing test suites [2].

However, these results mostly evaluate a program at a fixed point in time. Modern coding agents repeatedly modify the same codebase as new requirements arrive. A test generated at an early stage is useful only if it continues to expose regressions introduced by later modifications. It is therefore unclear whether improvements in mutation score transfer to regression detection during long-horizon, agent-driven software evolution.

We study this question using SlopCodeBench, where a coding agent implements checkpoint C_1 and repeatedly extends its previous solution through $C_2, C_3, \ldots$ [3]. Our main research questions are:

1. **RQ1:** Do mutation-guided tests detect more later-checkpoint regressions than the original tests or standard LLM-generated tests under the same test-generation budget?
2. **RQ2:** Does mutation score provide predictive value for future regression detection beyond statement and branch coverage?
3. **RQ3:** Which mutation operators and task domains produce useful regression tests, and when does mutation-guided testing fail to provide additional value?

B. Challenges

A first challenge is obtaining a reliable regression oracle while preventing information leakage. At checkpoint C_i, the official tests must be split into a visible set and a held-out oracle set. The held-out tests are never shown to either LLM test-generation method and are used only to confirm later regressions.

Another challenge is distinguishing meaningful mutants from uninformative ones. Equivalent, unreachable or dead-code, invalid, and out-of-contract mutants should not distort the main mutation score.

Finally, regression detection can be confounded by test quantity and code coverage. A method may appear stronger simply because it generates more tests or executes more code. We therefore keep the model, test-generation budget, and execution environment fixed and analyze whether mutation score adds information beyond statement and branch coverage.

C. Approach

We will use a pinned version of SlopCodeBench, which contains multi-checkpoint software tasks with checkpoint specifications, reference implementations, and checkpoint-specific tests. Later checkpoints extend the same workspace, naturally exposing regressions caused by later edits [3].

To isolate test quality from agent architecture, the primary experiment will use a single repository-level coding agent, such as Claude Code, with the model, agent version, prompt, and execution environment fixed. The agent trajectory will be generated first and then frozen so all test-generation methods evaluate the same later code changes.

We will initially restrict the experiment to Python tasks and use mutmut for mutation testing. Mutmut integrates with pytest and can mutate only covered source lines [4]. Coverage will be measured using statement and branch coverage.

At each eligible checkpoint C_i, the official tests are split into a visible set and a held-out oracle set. We compare three test suites:

1. **Original:** the visible SlopCodeBench tests only.
2. **Vanilla LLM:** the original tests plus at most N black-box tests generated from the specification and current code.
3. **Mutation-Guided:** the original tests plus at most N tests generated using surviving mutants as additional guidance.

For the mutation-guided method, the visible suite is executed against mutants generated by mutmut. A

surviving mutant represents a potential testing blind spot. The LLM receives the specification, relevant source context, and mutant difference and generates a test that passes the correct implementation but fails on the mutant.

Generated tests are retained only if they (1) execute successfully, (2) pass the reference implementation and current checkpoint implementation, (3) kill the target mutant when applicable, and (4) remain stable across repeated runs. Equivalent, unreachable, invalid, and out-of-contract mutants are excluded from the main mutation score. Results will also be reported by mutation-operator family and task category.

D. Evaluation

A confirmed regression occurs when a held-out prior-checkpoint test passes on the earlier correct implementation but fails after a later agent modification. The held-out suite therefore acts as an independent regression oracle.

Our primary metric is regression detection:

$$\text{Regression Detection Rate} = \frac{\#\text{confirmed regression events detected by the suite}}{\#\text{confirmed regression events}}.$$

Our secondary metric is mutation score:

$$\text{Mutation Score} = \frac{\#\text{killed non-equivalent valid mutants}}{\#\text{non-equivalent valid mutants}}.$$

All methods use the same model, maximum number of generated tests, and token budget. We then perform a coverage-controlled analysis using statement coverage, branch coverage, mutation score, and test count to determine whether mutation score adds information beyond simply executing more code [1].

We consider the method successful if mutation-guided tests improve later regression detection over the Original and Vanilla LLM baselines under the same budget, or if mutation score provides additional predictive value after controlling for coverage.

A negative result is also informative. If mutation score improves without improving regression detection, synthetic fault detection may not transfer well to regressions introduced by long-horizon coding agents. If the improvement disappears after controlling for coverage, broader code coverage is likely responsible. Benefits limited to particular operator families or task domains will indicate where mutation testing is most useful.

E. Timeline and Task Assignments

Period	Activities
Sep. 18–27	Pin SlopCodeBench version; integrate agent, mutmut, and coverage.py; run pilot mutation analysis.
Sep. 28–Oct. 18	Generate and freeze agent trajectories; implement mutant filtering, test generation, and oracle splitting.
Oct. 19–Nov. 8	Run Original, Vanilla LLM, and Mutation-Guided experiments under equal budgets.
Nov. 9–22	Perform coverage-controlled, operator-level, domain-level, and failure analyses.
Nov. 23–Dec. 9	Complete final experiments, report, presentation, and code release.

Hengyu Zhang: LLM test generation, mutant validation, equivalent-mutant analysis, and domain/operator analysis.

Shipeng Zhao: SlopCodeBench and agent integration, mutmut/coverage pipeline, trajectory collection, and regression evaluation.

Both members will jointly design experiments, analyze results, and prepare the final report and presentation.

F. Deliverables

By the end of the project, we expect to deliver: (1) a working mutation-guided regression-testing pipeline; (2) generated test suites and frozen coding-agent trajectories; (3) an experimental comparison of Original, Vanilla LLM, and Mutation-Guided testing; (4) coverage-, operator-, and domain-level analyses; and (5) the final project report, presentation, and released code.

References

[1] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are Mutants a Valid Substitute for Real Faults in Software Testing?" *ACM SIGSOFT FSE*, 2014.

[2] C. Foster, A. Gulati, M. Harman, I. Harper, K. Mao, J. Ritchey, H. Robert, and S. Sengupta, "Mutation-Guided LLM-based Test Generation at Meta," *arXiv:2501.12862*, 2025.

[3] G. Orlanski, D. Roy, A. Yun, C. Shin, A. Gu, A. Ge, D. Adila, F. Sala, and A. Albarghouthi, "SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks," *arXiv:2603.24755*, 2026.

[4] "mutmut: Python Mutation Tester," project documentation, `mutmut.readthedocs.io`.